

Vibecoding: открытые вопросы

AI-агенты и инструменты

1. Чем **агент** отличается от **AI-чата**?
2. Почему ChatGPT, написавший код, ещё не даёт готовый продукт?
3. Что такое **vibecoding**?
4. Какие есть три категории инструментов (чат, редактор-агент, сборщик приложений) и чем они отличаются?
5. С какого типа инструмента новичку проще начать и почему?
6. Почему хакатон Anthropic выиграли не профессиональные разработчики?
7. В чём была главная причина, по которой AI убрал барьер для не-программистов?

Поиск и проверка идеи

1. Почему выдуманные “за столом” идеи почти всегда проваливаются?
2. Какие есть источники хороших идей (боль, сила, грязная работа и т.д.)?
3. Почему хорошая идея часто выглядит немного безумной?
4. Почему лучше целиться в узкую группу пользователей, а не в широкую?
5. Какие вопросы стоит задать себе, чтобы быстро проверить идею?
6. Почему AI и близкие люди почти всегда хвалят твою идею?

7. Каким промптом превратить AI из “да-человека” в критика?
8. Какие правила у метода **The Mom Test**?
9. Чем плохой вопрос клиенту отличается от хорошего?
10. Какие есть способы проверить идею на практике (кроме разговоров с людьми)?
11. Какая самая частая причина провала стартапов?
12. Какие книги стоит прочитать про поиск и проверку идей, и в каком порядке?

Риски и грабли vibecoding

1. Что происходит, если API-ключ оказывается прямо в коде и попадает в публичный репозиторий?
2. Где должны храниться секреты и почему?
3. Зачем ставить лимит трат на платных сервисах?
4. Что может пойти не так, если слепо доверять коду от AI без проверки безопасности?
5. Что такое оверинжиниринг и чем он опасен в vibecoding?
6. Как понять, что задача решена “эффективно”, а не переусложнена?
7. В каких сценариях vibecoding подходит хорошо, а в каких — опасен?
8. Что имеется в виду под “проблемой последних процентов” в AI-разработке?
9. Из чего складывается стоимость vibecoding (подписки, токены, итерации)?
10. Почему агенты тратят намного больше токенов, чем обычный чат?

Промптинг

1. Из каких частей состоит хороший промпт?

2. Чем плохой промпт отличается от хорошего на конкретном примере?
3. Какие техники промптинга реально работают (конкретность, шаги, chain-of-thought и т.д.)?
4. Как правильно возвращать AI информацию об ошибке, чтобы он её починил?
5. Что делать, если AI повторяет одну и ту же ошибку раз за разом и не может решить задачу с первого раза (застрял в цикле)?
6. Почему после нескольких неудачных попыток стоит остановиться, а не продолжать долбить тот же промпт?
7. Почему свежий чат иногда решает проблему, которую не решает продолжение старого?

Базовые технические знания

1. Почему важно понимать Git, серверы и Docker, если весь код пишет AI?
2. Что такое Git и зачем он нужен при работе с AI-агентом?
3. Зачем делать коммит перед каждым большим изменением?
4. Зачем нужны тесты, если код и так “вроде работает”?
5. Как AI-агент может сам себя чинить с помощью тестов?
6. С какого количества тестов стоит начинать проект?
7. Что такое Docker-контейнер простыми словами?
8. Какую проблему решает Docker (“у меня на компе работало”)?
9. Чем домен отличается от хостинга?
10. Что значит “задеплоить” приложение?

Хранение кода и проекта

1. Где должен храниться код проекта, чтобы был бэкап и история изменений?
2. Зачем нужен GitHub (или аналог), если Git уже есть локально?

3. Что дают ветки (branches) в Git при экспериментах с кодом?

Прокачка AI и повторяющиеся инструкции

1. Как избежать повторения одних и тех же инструкций AI в каждом новом чате?
2. Чем отличаются CLAUDE.md, AGENTS.md и SKILL.md?
3. Что такое MCP и зачем подключать AI к своим данным?
4. Какие примеры коннекторов MCP можно использовать в реальной работе?

Как приходят идеи и как их формулировать для AI

1. Как аналитик может использовать свои навыки (BR, SRS, FR, NFR) при работе с AI-инструментами?
2. Что такое spec-driven development и почему оно решает проблему размытых промптов?
3. Как AI помогает генерировать диаграммы (UML, BPMN) из текстового описания?

Дисциплина и правила

1. Какие golden rules должен соблюдать вайбкодер, чтобы не создать проблему вместо продукта?
2. Почему решения про деньги и безопасность должен принимать человек, а не AI?
3. Когда стоит звать живого разработчика вместо того, чтобы продолжать вайбкодить самому?